GRYNX

TRACK · SYNC · EXECUTE

Production Floor Management System

Functional & Technical Specification

A complete reference for GRYNX by D-LYFT — what it does, how every screen works and what we call each part in code, the backend/frontend architecture and design system, its strengths and weak points, and where it can go next. Detailed enough to rebuild the system from scratch.

D-LYFT · GRYNX v0.2.0 · 2026-06-10

1 · What GRYNX is

GRYNX is a **least-human-interjection** production-floor management system for a fabrication factory (trusses, stage, scaffolding and similar). Its founding principle: **the floor should barely touch the app**. Work moves by **scanning a barcode job card** at each station — a scan automatically marks the previous step complete and places the job at the scanning station. Operators don't press "done", don't pick statuses, don't fill forms; they scan a card as it arrives, and occasionally answer an update request. Everything else (planning, approval, traceability, maintenance) is captured around that single gesture.

A sales order becomes a **PPC request**; the admin approves it into a **job** with a printable **job card**; **Purchase** logs the raw material; the job flows through the department **pipeline** by scanning, through **QC** and **FG Stock** (which assigns serial numbers), until it is **closed**. Every unit stays permanently searchable by job, serial, date and raw-material batch. **Maintenance** runs alongside: anyone raises a ticket, the crew is alerted and chases it to closure.

1.1 · The production flow

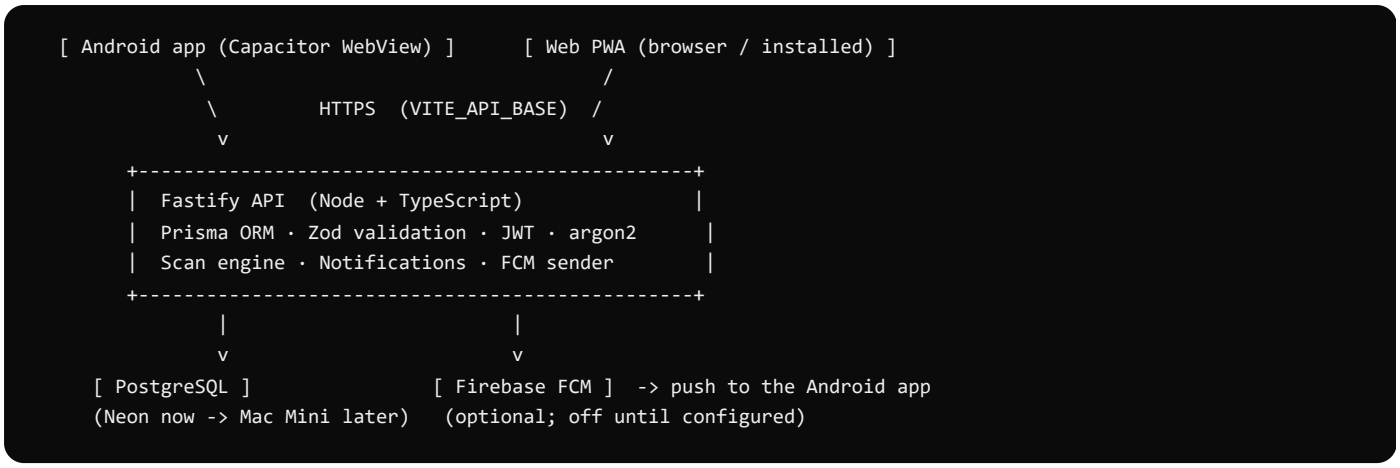
```
SALES ORDER
   |
   v
PPC raises REQUEST  --(Admin: Approve / Request-Change / Edit -> PPC confirm)-->  JOB CREATED  +  printable job card
   |
   v
PURCHASE logs raw material   (item . vendor . batch)            <-- start of genealogy
   |
   v   every station SCANS the card on arrival  ->  job advances, previous step auto-completed
DESIGN -> LASER -> ALLOY/MS PROD -> CNC/VMC -> POWDER -> QC --approve--> FG STOCK
                                                                          |  assign serial numbers
                                                                          v
                                                                  request closure --Admin approve--> CLOSED

MAINTENANCE  (any user raises a ticket -> crew alerted -> assigned -> resolved & closed)   runs in parallel.
TRACEABILITY: every unit is searchable by  job card / serial / date / raw-material batch.
```

1.2 · Roles — who does what

ROLE	EXAMPLE USER	LANDS ON	RESPONSIBILITY
Admin	AASHISH	Admin Home	Approve PPC requests, view any job's full history, manage departments & maintenance
PPC (Planning)	Deepak	PPC Request	Convert sales orders into production requests for approval
Dept Head / Floor operator	per department	Station Home	Scan job cards to advance them through the pipeline
QC	qc	QC Inspection	Approve to FG, or send for rework
FG Stock	fg	FG Stock	Assign serial numbers, request closure
Purchase	purchase	Purchase	Log raw material (item · vendor · batch) per job
Maintenance crew	maint	Maintenance	Get alerted, assigned, and resolve machine/facility tickets

Every user signs in with a Login ID + 6-digit PIN (biometric optional). Roles are assigned per user (a user can hold several). An internal "Administrator" test account can preview every screen, but it is not a factory role.

2 · System architecture



2.1 · Frontend

- **Stack:** React 18 + TypeScript + Vite, shipped as an installable **PWA** and wrapped with **Capacitor** into a native Android app (same codebase, bundled offline shell).
- **Routing:** a `Screen`-based router in `App.tsx` (a switch, no URL router). A `navTo()/registerNav()` singleton lets any component navigate.
- **API client:** `src/lib/api.ts` — typed, with token storage + transparent JWT refresh, and a DEMO fallback (`src/lib/demo.ts`) that runs entirely in the browser with no server.
- **Native bootstrap:** `src/lib/native.ts` (status bar, splash, FCM token registration, notification channel) — a no-op on the web build.
- **Scanning:** html5-qrcode (camera) with a manual-entry fallback. **Biometrics:** WebAuthn (Face-ID/fingerprint).

2.2 · Backend

- **Stack:** Fastify 5 + Prisma 6 + Zod + @fastify/jwt + argon2, on PostgreSQL.
- **Auth:** PIN/password → argon2 verify → short-lived access JWT + refresh token; per-username throttle; biometric login via WebAuthn (domain-bound).
- **Scan engine** (`/scan`): the core. An **arrival scan** advances a job — idempotency keys (unique) prevent double-apply, optimistic `version` locks prevent concurrent races, and the station is derived from the signed-in user. Single write authority.
- **Job creation:** opaque job number + daily sequence + a snapshot of the pipeline steps; the first department is notified.
- **Notifications:** written to the DB (the in-app bell) and fanned out to **FCM push** as one source of truth, fire-and-forget.
- **Modules:** jobs, ppc (request round-trip), qc, fg (serials + closure), purchase (materials + batch genealogy), maintenance (ticket lifecycle), notifications, devices (FCM tokens), webauthn.

2.3 · Core data model (selected)

- **User** — roles, PIN/password hash, webauthn credentials, push tokens.
- **Job** — product, models, status, dates; **JobStep** — per-department step (status, accepted/completed timestamps, version).
- **PpcRequest** — request + status round-trip (submitted/clarification/pending_confirm/approved).
- **MaterialUsage** — raw material per job (vendor, batch).
- **Serial** — FG serial per unit.
- **MaintenanceTicket** + **MaintenanceEvent** — ticket + thread + photo.
- **Notification** — per-user alert (type, links, read state).
- **ScanEvent** — the audit of every scan.

2.4 · Hosting

Interim: API on Render (free tier) + PostgreSQL on Neon; web on Vercel. Endgame: a **Mac Mini on the factory LAN** (Postgres + API) exposed via a **Cloudflare Tunnel** for remote access — devices on-site hit it in milliseconds, off-site through the tunnel. Moving servers is a connection-string + `VITE_API_BASE` change, no rebuild.

3 · UI terminology — what we call each part (in code)

Every screen is built from the same scaffold. These are the names we use in conversation and the matching classes/components in the code.

WE CALL IT	IN CODE	WHAT IT IS
App shell	`.app`	Full-height flex column (100dvh) that locks the viewport; holds header, body, footer.
Header	`TopBar · .ubar--top`	Logos, the signed-in user's identity, the notification Bell (with unread badge), and Lock/sign-out.
Body	`.app__body`	The active screen's content. The shell is fixed; only the body's inner list (.screen__scroll) scrolls.
Footer / status bar	`BottomBar · .ubar--bottom`	App version (left), live connection pill ONLINE/OFFLINE (centre), ENCRYPTED·TLS (right).
Screen	`.screen / .jobscreen`	A page. .screen__head = back button + titles; .screen__scroll = the scrolling content.
Form	`JobForm · .jobform`	The reusable job/request form (product, models table, priority, pipeline, schedule).
Popup / modal	`.mnt__overlay+.mnt__modal, JobCardModal, .qpop, BiometricEnroll`	Overlays for raise-ticket, log-material, add serials, add-model, print card, enrol biometrics.
Row / card	`.navrow .jrow .ppcq__card .dh__row .ticket .notif`	Tappable list items across the modules.
Chip / tag / badge	`.chip .pri-tag .dh__tag .jstatus .notif__cat`	Status pills, priority tags, and the notification category chips.
Button	`.btn (--solid / --primary / --ghost / --danger / --block)`	The button system; "hero" buttons (e.g. Scan) are larger and accented.
Stepper	`.stepper / .step`	The pipeline progress indicator on a job/ticket.
Router	`App.tsx switch on Screen + navTo()/registerNav()`	A screen-based router (no URL routing) — a singleton lets any component navigate.

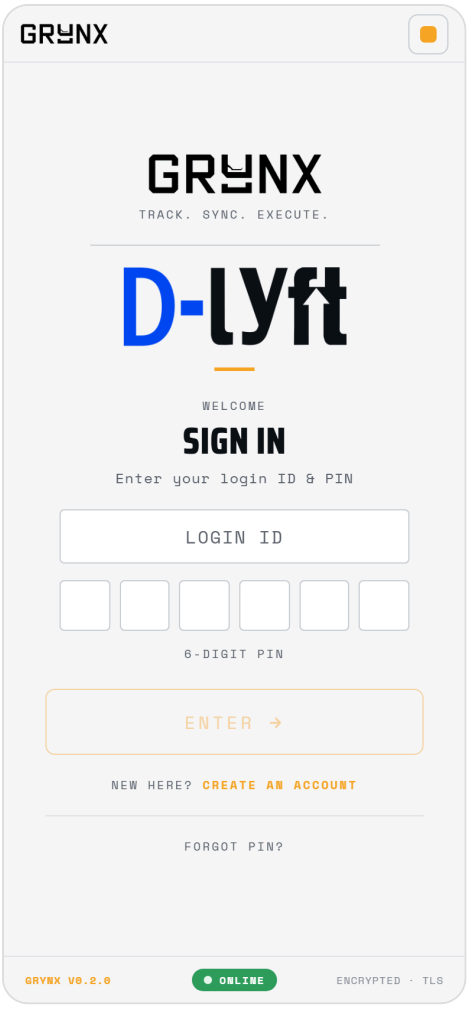
Design language: amber `#F5A623` accent (`--brand`) on a Day (light) or Night (dark) theme; display type = Saira Condensed, mono/labels = Space Mono. All interactive elements use the accent; six selectable accent presets exist. The whole UI is sized to the device once on load (a viewport-fit scaler), so it fits any phone.

EVERYONE

1. Login

Entry screen. PIN-first so floor users sign in fast; office users may use a password. Biometric (WebAuthn / Face-ID / fingerprint) appears once enrolled on the device. Login ID is case-insensitive and remembered.

Code: LoginPage · .login · **Data:** POST /auth/login → JWT (access+refresh)



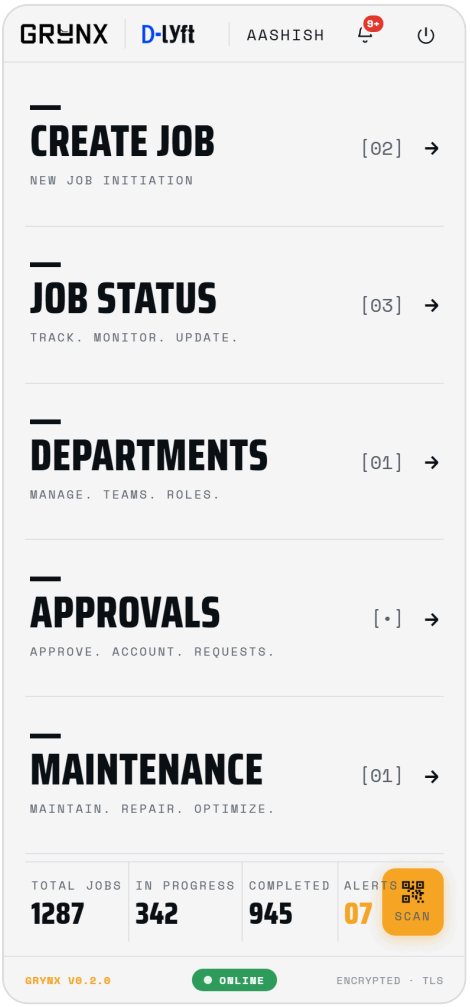
1. **Header (TopBar, theme variant)** — On login it carries only the brand + the accent/Day-Night theme picker.
2. **GRYNX wordmark + tagline** — “Track. Sync. Execute.” — the product brand.
3. **Login ID field (.login__user)** — The username; remembered to localStorage for next time.
4. **PIN boxes (.pin__box ×6)** — 6-digit PIN; each digit masks the instant the next is typed (iOS-style).
5. **Enter (.login__enter)** — Submits → routed to the role’s home by landingFor().
6. **Biometric button (.login__bio)** — Passwordless sign-in via the platform authenticator (shown only for an enrolled, returning user).

ADMIN (AASHISH)

2. Admin Home

The factory admin's control centre — a list of modules, a live stats bar, and a scanner. Deliberately sparse: the admin manages and approves; the floor drives jobs by scanning.

Code: AdminHome · .admin · **Data:** static nav + (stats are placeholder counts)

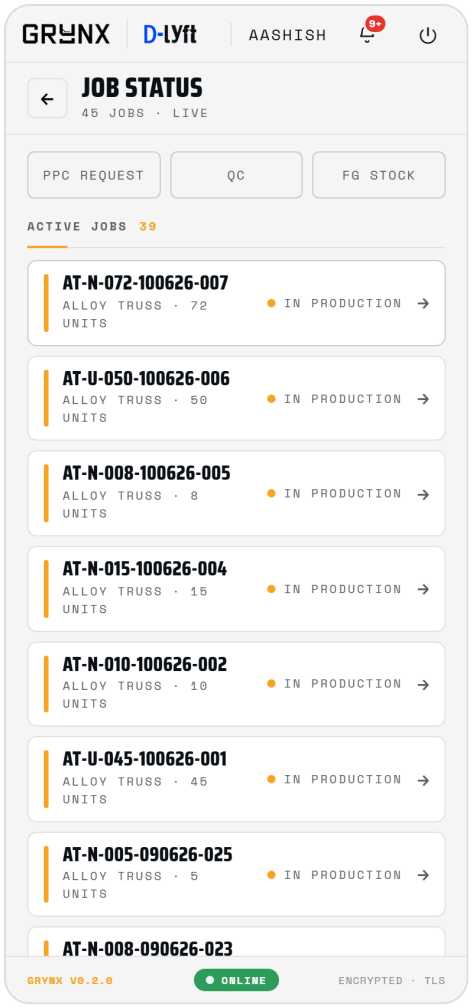


1. **Header (TopBar, .ubar--top)** — Logos · user identity (name · role · ID) · notification Bell with unread badge · Lock/sign-out.
2. **Navigation rows (.navrow)** — Create Job, Job Status, Departments, Approvals, Maintenance — each routes to a module.
3. **Stats bar (.admin__statsbar)** — Total Jobs / In Progress / Completed / Alerts. Tapping the stats opens the Admin Overview ("admin panel").
4. **SCAN button (.admin__scanbtn)** — Sits in the stats bar; scans any job card to open its full history (read-only lookup).
5. **Footer / status bar (BottomBar, .ubar--bottom)** — App version · live ONLINE/OFFLINE pill · ENCRYPTED·TLS.

ADMIN 3. Job Status

The live job board — every real job from the database, grouped by state. The admin's window into the whole floor.

Code: JobStatus · .screen / .jrow · **Data:** GET /jobs (live)

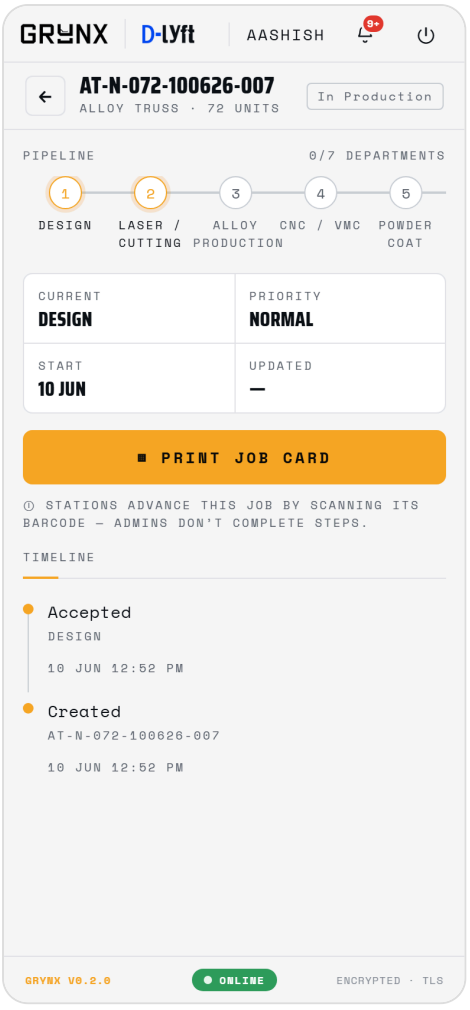


1. **Screen head (.screen__head)** — Back button + title + "N jobs · live".
2. **Stage shortcuts (.js__stages)** — Jump to PPC Request / QC / FG screens.
3. **Needs Attention / Active / Completed (.jsec)** — Sections grouping jobs by status; counts in the heading.
4. **Job row (.jrow)** — Priority bar + Job ID + product/qty + a status badge (.jstatus). Tap → that job's history.

ADMIN 4. Job History (Job Detail)

The complete genealogy of one job: where it is, who touched it and when, the raw material it's built from, and its finished serial numbers. This is the traceability spine.

Code: JobDetail · .jobdetail · **Data:** GET /jobs/:id + /purchase/:id/materials + /fg/:id/serials

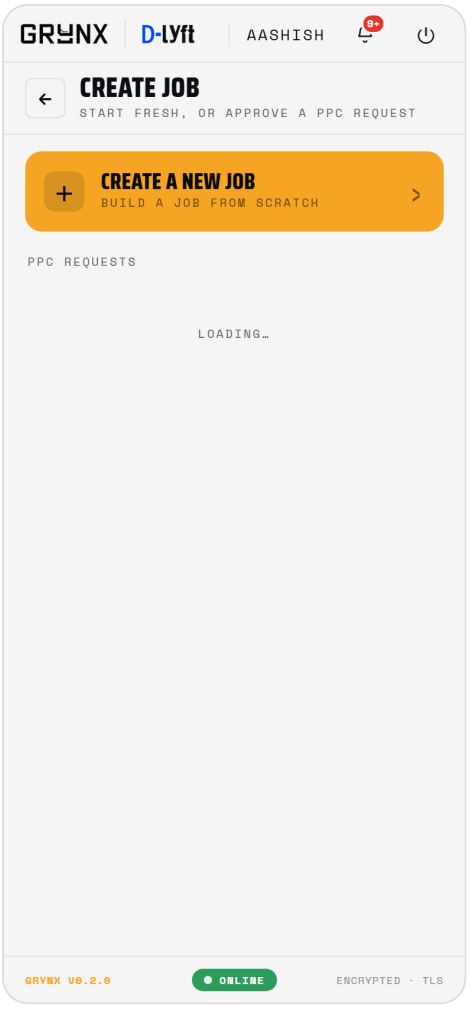


1. **Status chip (.chip)** — Live job status (In Production / In QC / Closed …).
2. **Pipeline stepper (.stepper / .step)** — The departments the job flows through; ticked = done, amber = current.
3. **Meta cells (.jd__meta)** — Current department, Priority, Start, last Update/Close date.
4. **Print Job Card (.btn)** — Opens JobCardModal (QR + Code-128 barcode card).
5. **Raw Material (.jd__matline)** — Item · vendor · batch logged by Purchase — the start of genealogy.
6. **Serial Numbers (.jd__serial)** — FG's serials for the finished units.
7. **Timeline (.timeline / .tl)** — Every event with a real timestamp.

ADMIN 5. Create Job (hub)

The job entry point. Start a fresh job, or pick up a request PPC already raised — review is the default, fresh creation is the exception.

Code: JobHub · .jobhub · **Data:** GET /ppc (submitted)

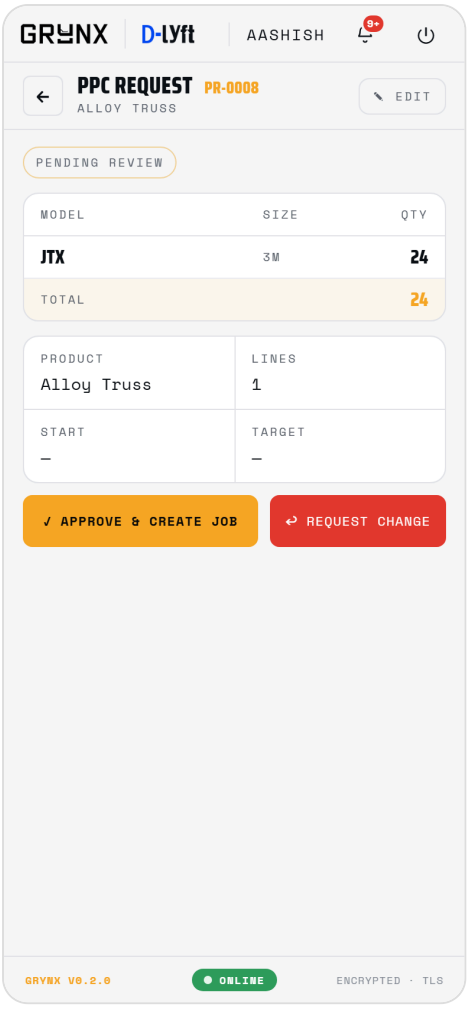


1. **+ Create a new job (.jobhub__new)** — Opens the blank Create Job form.
2. **PPC Requests list (.ppcq__card)** — Pending requests; tap one to review it as a job sheet.

ADMIN 6. PPC Request — Review (Job Sheet)

A PPC request opens as a read-only job sheet (not a form). Review first; edit only if needed. Edits don't apply unilaterally — they round-trip to PPC to confirm, for accountability.

Code: PpcReviewSheet · .prs · **Data:** POST /ppc/:id/approve|propose|request-change

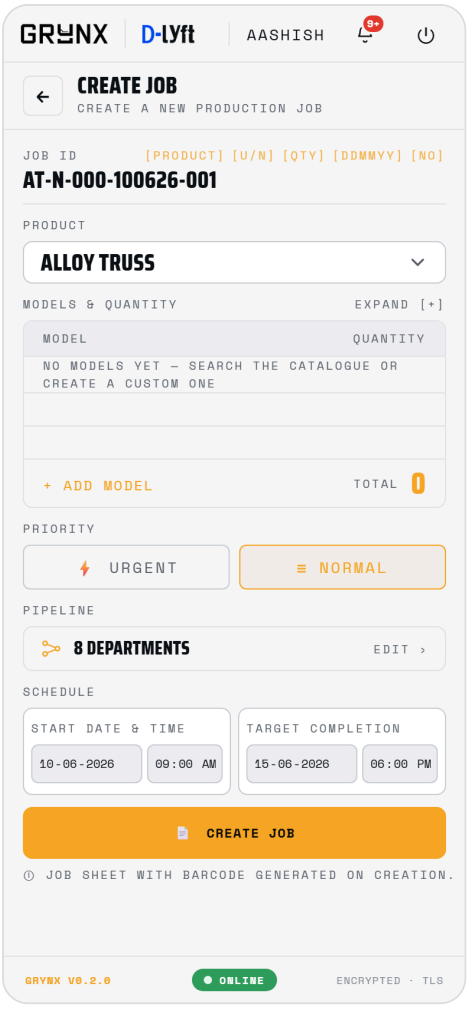


1. **Status chip + line table (.prs__sheet)** — Model · Size · Qty per line + total, with a status chip.
2. **Edit (.prs__edit)** — Flips the sheet into the form to modify (→ proposal back to PPC).
3. **Approve & Create Job** — Turns the request into a live job + a printable card.
4. **Request Change (RC)** — Sends it back to PPC with a note to fix & resubmit.

ADMIN / PPC 7. Create Job (form)

The shared production form. Admin “Create Job” makes a job directly; PPC “Submit Request” raises a request for approval. Same UI, different verb.

Code: CreateJob + JobForm · .jobscreen / .jobform · **Data:** POST /jobs (admin) · POST /ppc (PPC)



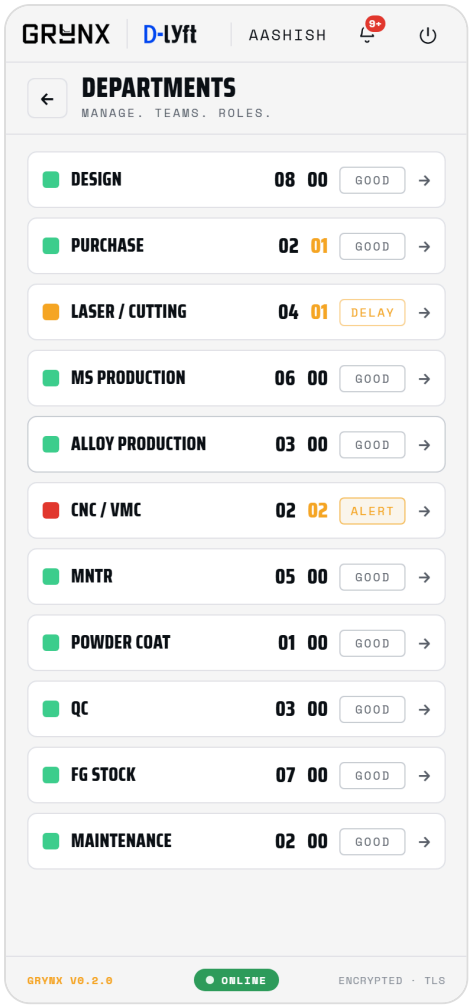
1. **Job ID (.jobid__value)** — Auto-built code preview (product · priority · qty · date).
2. **Product (CustomSelect)** — Product family (Alloy Truss, MS Truss …).
3. **Models & Quantity (.mtable)** — Add model types + sizes + quantities; a popup (.qpop) adds each line.
4. **Priority (.prio)** — Urgent / Normal.
5. **Pipeline (.pipeline)** — The ordered departments (editable per job).
6. **Schedule (.sched)** — Start date/time + target.

ADMIN

8. Departments

A heat-map of every department and its load. Tap to drill into a department's queue and team.

Code: Departments · .screen · **Data:** GET /departments



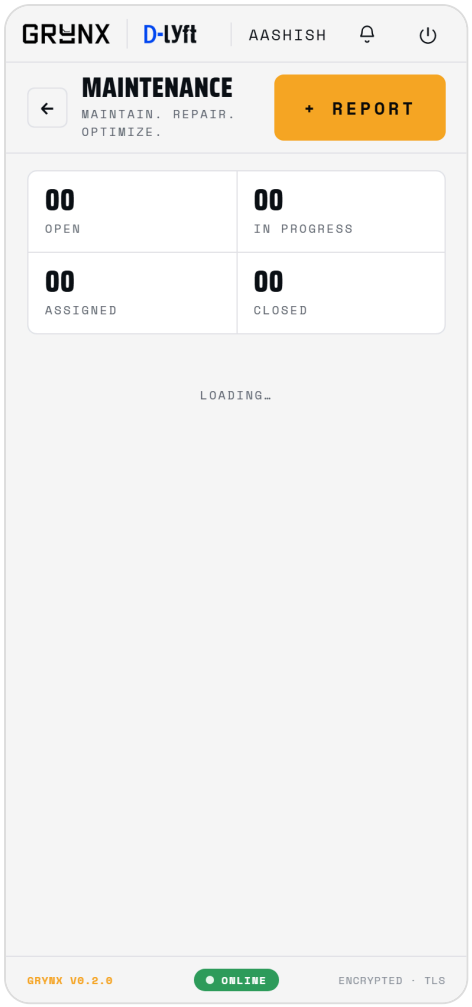
1. **Department row** — Name + live load indicator; tap to open.

EVERYONE RAISES · CREW/ADMIN MANAGE

9. Maintenance

Raise and track machine/facility issues. Aggressive by design: notify the crew immediately and chase to resolution.

Code: Maintenance · .mnt · **Data:** GET/POST /maintenance



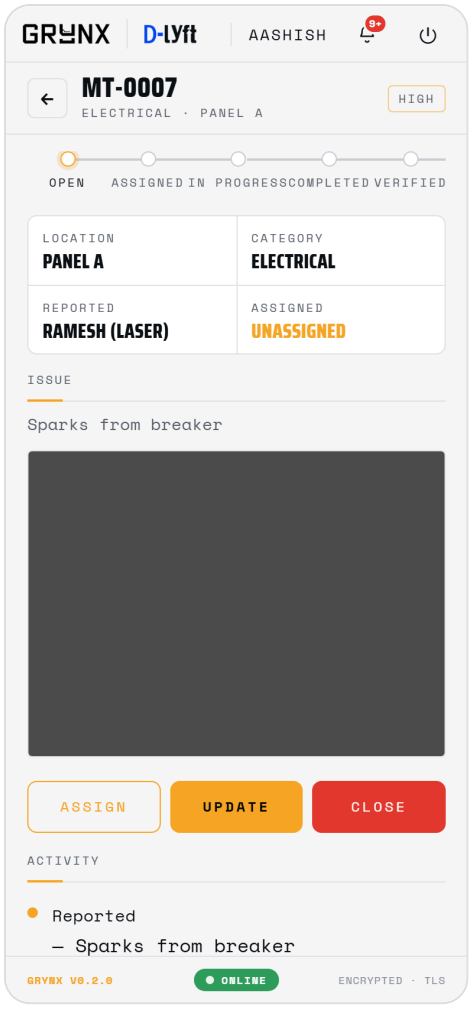
1. **Report an Issue (.mnt__report)** — Opens the raise popup (category, priority, location, description, optional photo).
2. **Ticket list (.ticket)** — Location/machine · ticket no · category · assignee · priority.
3. **Stats (.mnt__stat)** — Counts by state.

MAINTENANCE CREW / ADMIN

10. Maintenance Ticket

One ticket's full lifecycle with a photo and a thread — assign a tech, log ETA/parts, close with a remark.

Code: MaintenanceDetail · .md · **Data:** GET /maintenance/:id · POST assign|update|close



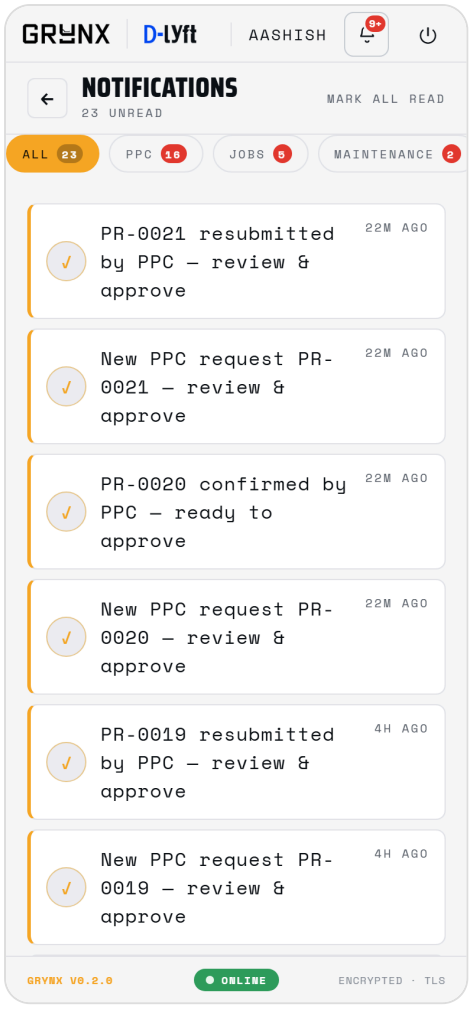
1. **Status stepper** — open → assigned → in progress → closed.
2. **Issue + photo (.md__photo)** — The reported fault and the photo of it (client-compressed).
3. **Assign / Update / Close (popup)** — Head assigns; tech logs ETA + parts; head closes with a remark.
4. **Timeline (.timeline)** — Threaded history.

EVERYONE

11. Notifications

The alert feed, now bifurcated into categories so it isn't one undifferentiated list. Tapping a notification opens exactly its source.

Code: Notifications · .notif · **Data:** GET /notifications · /count

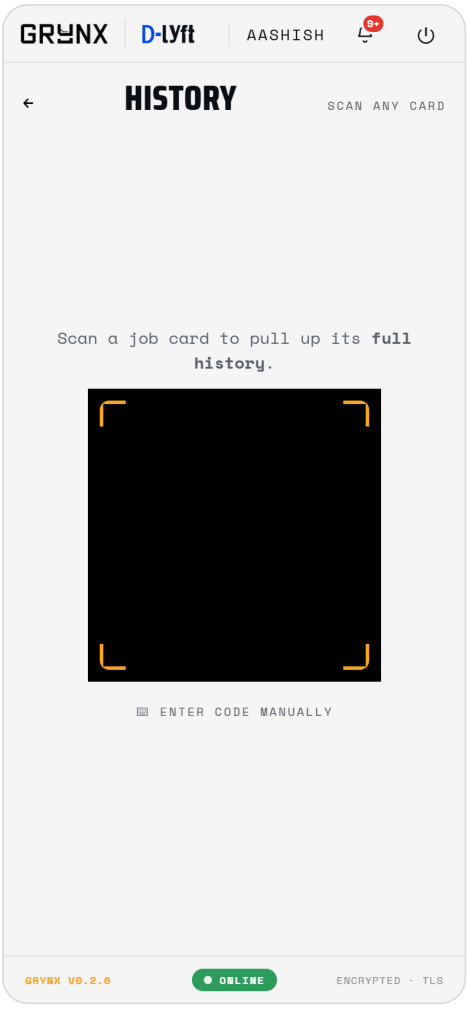


1. **Category chips (.notif__cat)** — All / PPC / Jobs / Maintenance, each with a per-category unread count; filters the feed.
2. **Notification (.notif)** — Tap → its source: a PPC request, a printable job card, or a maintenance ticket.
3. **Mark all read (.notif__readall)** — Clears unread.

ADMIN 12. Admin Scan → History

Reached from the stats-bar SCAN button. The camera opens immediately; scanning a card resolves it to its full history (no advance).

Code: ScanPage (mode=lookup) · .scan · **Data:** GET /jobs/lookup?code



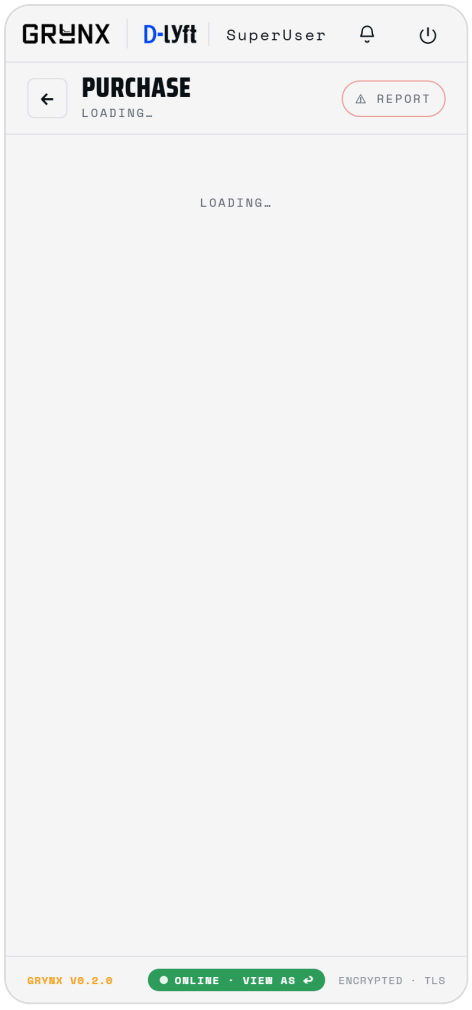
1. **Viewfinder (.scan__camwrap / .scan__frame)** — Small framed camera window; point at the card's QR/barcode.
2. **Enter code manually (.scan__manual-link)** — Type the code; the camera stops; tap outside to return to it.

PURCHASE

13. Purchase

Logs the raw material each job is made from — the first link in the traceability chain.

Code: PurchaseHome · .screen / .dh · **Data:** GET/POST /purchase/:id/materials



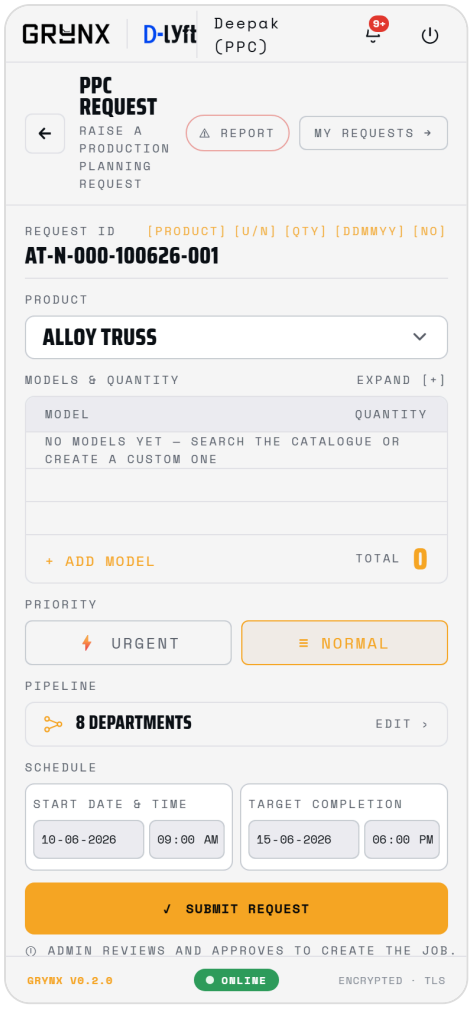
1. **Active jobs (.dh__row)** — Jobs needing raw material; badge shows how many materials are logged.
2. **Log raw material (popup)** — Item · type · vendor · batch · quantity, recorded against the job.

PPC (DEEPAK)

14. PPC Request

The same form as Create Job, but it submits a request the admin reviews. PPC never creates a job directly.

Code: CreateJob (variant=ppc) · **Data:** POST /ppc · GET /ppc/mine

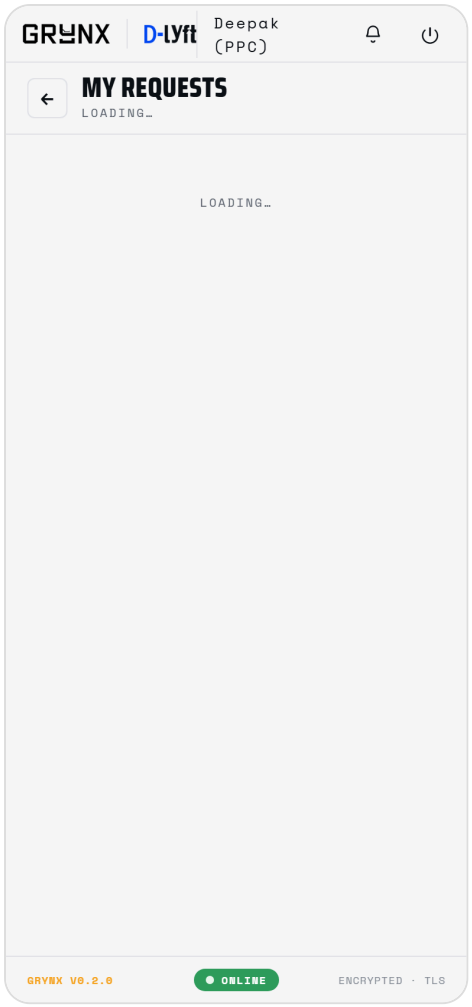


1. **Submit Request** — Sends it to the admin (no job yet).
2. **My Requests (.jobscreen__pill)** — Opens the PPC inbox of raised requests.

PPC 15. PPC Inbox (My Requests)

Where PPC acts on the admin's response — confirm proposed edits, or fix & resubmit after a Request-Change.

Code: PpcInbox · .ppcq · **Data:** GET /ppc/mine



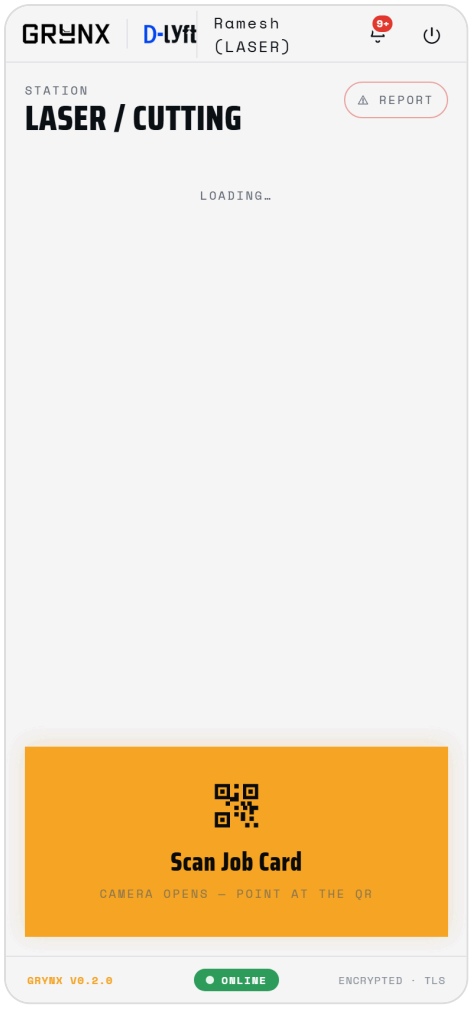
1. **CONFIRM** — Admin proposed edits — confirm to send back for approval.
2. **FIX** — Admin requested a change — edit and resubmit.
3. **WITH ADMIN** — Submitted, awaiting the admin.

FLOOR / DEPT HEAD

16. Station Home

A production station's screen. The operator sees the jobs heading their way and scans them in — almost no other interaction.

Code: StationHome · .station · **Data:** GET /jobs/queue

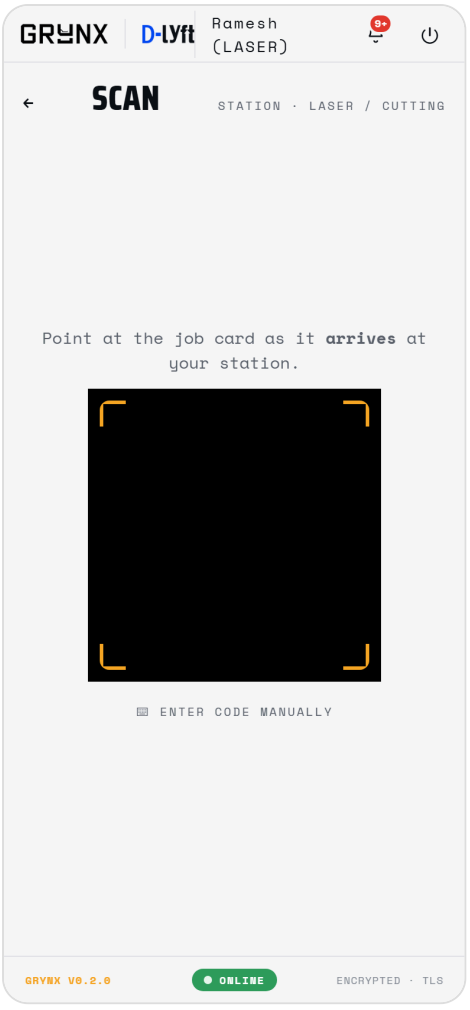


1. **Station name (.station__title)** — Which station you operate (from your role).
2. **At station / Incoming (.qjob)** — Jobs in progress here + jobs arriving next.
3. **REPORT (.station__report-btn)** — Raise a maintenance issue from here.
4. **Scan Job Card hero (.station__scan--hero)** — Big bottom button — opens the camera to scan a card on arrival, advancing the job.

FLOOR 17. Scan (advance)

The arrival scan — the heart of the system. Scanning a card moves the job to your station and auto-marks the previous step done.

Code: ScanPage (mode=advance) · .scan · **Data:** POST /scan

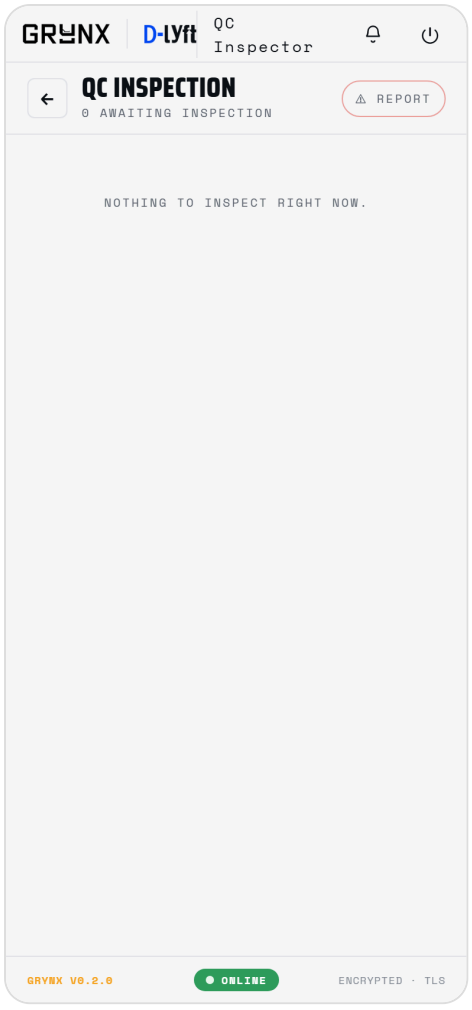


1. **Viewfinder** — Point at the card as it ARRIVES.
2. **Confirm (.scan__confirm)** — Review the move (completes X → now at Y) and confirm.
3. **Result (.scan__result)** — Advanced / duplicate / out-of-sequence, with a supervisor force-advance option.

QC 18. QC Inspection

The quality gate. Approve a job to release it to FG, or send it for rework.

Code: QcHome · .dh · **Data:** GET /qc/queue · POST approve|rework



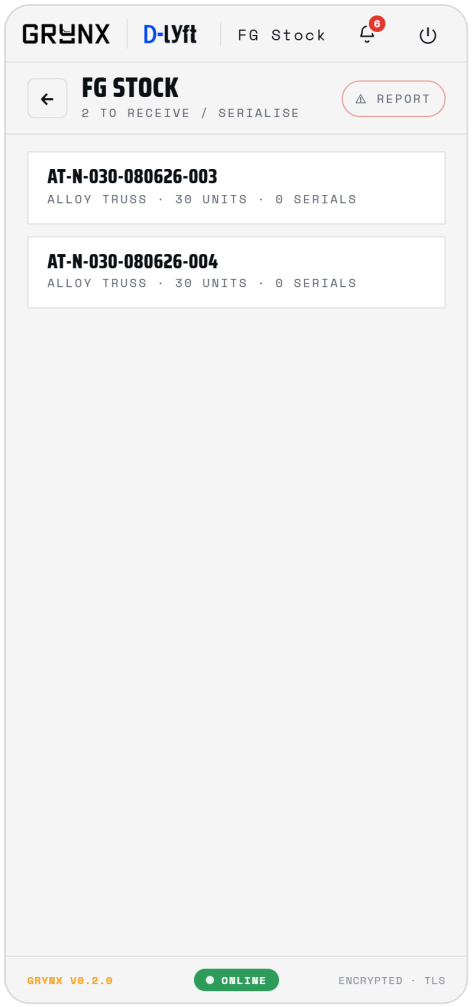
1. **Queue (.dh__row)** — Jobs at QC awaiting inspection.
2. **Approve / Rework (popup)** — Approve arms FG; rework flags a problem.

FG STOCK

19. FG Stock

Finished goods. Assign serial numbers and request closure; the admin approves to close the job.

Code: FgHome · .dh · **Data:** GET /fg/queue · POST serials|closure



1. **Queue** — Jobs received at FG.
2. **Add serial numbers (popup)** — The serials for the finished units.
3. **Request closure** — Asks the admin to approve & close.

4 · Design theory

- **One gesture.** The floor's entire interaction is "scan the card on arrival." Every screen for a floor role is built around making that one action fast and unmistakable (a big Scan hero, a short queue, nothing else).
- **Read before edit.** Approvals open as read-only sheets; editing is a deliberate secondary action that round-trips for accountability. Defaults favour the common path.
- **Single write authority.** State changes go through one engine (scan / explicit endpoints) with idempotency + version locks — no two clients can corrupt a job's state.
- **Opaque codes, human labels.** The barcode encodes an opaque job number; people see a readable Job ID. The two are linked but independent.
- **Same codebase everywhere.** Web PWA and Android share one React build; native features degrade to no-ops on the web.
- **Visual hierarchy.** Condensed display type for identity/headings, mono for data/labels, amber only for what you can act on, status carried by colour + a dot.

5 · Strengths & weak points

Strengths

- Genuinely low-friction for the floor — scan-driven, minimal training.
- End-to-end traceability (raw batch → job → serial → date) is built into the data model, not bolted on.
- Robust state engine (idempotent, optimistic-locked) — safe under concurrent scans.
- One codebase → web + Android; cheap to host; portable (Neon → Mac Mini with no rewrite).
- Full audit trail + per-user notifications with push.

Weak points / risks (to address before scale)

- **No offline scan queue yet** — a Wi-Fi blip blinds a station. #1 floor risk; scans must queue locally and sync.
- **Single server = single point of failure** — the Mac Mini holds the factory's whole memory. Needs UPS + automated off-site backups before real data.
- **Maintenance escalation timer not built** — thresholds exist (15-min acknowledge / 30-min assign) but nothing fires them yet; an ignored ticket doesn't auto-escalate.
- **Shared PINs in pilot** — move to per-user PINs; add a supervisor gate on force-advance.
- **Some admin surfaces still placeholder** — Admin Home stat numbers, Overview, Insights, Departments are not yet wired to live counts.
- **WebAuthn in the Android WebView** may be flaky — keep a native biometric plugin as a fallback.

6 · Suggestions & future integrations

- **Offline-first scan queue** (the priority).
- **Maintenance escalation timer** (acknowledge/assign breaches → notify head + admin).
- **LAN-first endpoint race** so one APK auto-picks the Mac Mini on-site, tunnel off-site.
- **Label/job-card printing** to a network thermal printer.
- **Split / merge job cards** (parallel sub-cards that converge) — schema already anticipates it.
- **ERP / Tally / accounting** export of closed jobs + material consumption.
- **Analytics & SLA dashboards** (throughput, bottlenecks, on-time %).
- **Photo storage to object store (R2/S3)** instead of base64 in the DB.
- **Barcode-gun support** on the desktop panel (keyboard-wedge input).
- **Role-based dashboards** + shift handover summaries.

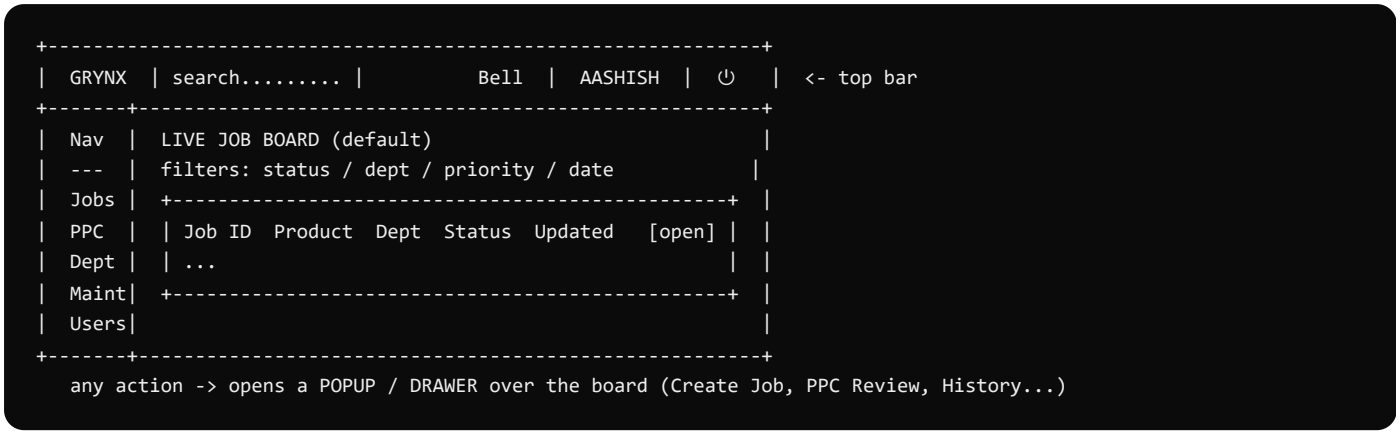
7 · Desktop Admin Panel (specification)

A separate, desktop-optimised admin surface — **everything the phone app can do for the admin, but for a big screen and a keyboard**. It reuses the same API and TypeScript client; only the layout differs. Built as a responsive desktop layout (or a second Vite entry) over `src/lib/api.ts`.

7.1 · Shape: single page + popups

It is **mostly one screen**. A persistent left rail (or top bar) holds navigation + global search; the centre is the **live job board** (the Job Status data) as the default working surface. Every action opens as a **popup/modal or side drawer** over that board — the admin never loses context:

- **Create Job** → popup form.
- **PPC Review** → popup job sheet (Approve / RC / Edit).
- **Job History** → side drawer (timeline + materials + serials).
- **Departments** → popup with the heat-map + drill-in.
- **Maintenance** → popup list + ticket detail.
- **Notifications** → drawer, bifurcated (PPC / Jobs / Maintenance).
- **Approvals / Users** → popup.
- **Scan / lookup** → a search box + barcode-gun (keyboard-wedge) input — no camera needed.



7.2 · How it differs from the phone app

- Wider, multi-column, information-dense; bulk actions + powerful filters/search.
- Keyboard-first; barcode gun instead of a camera for lookup.
- Same data + permissions + API — no separate backend. Reuses the screen components as modal bodies where possible.
- Intended for the admin's office desk; the phone app remains the floor + on-the-go surface.

Build note: the fastest route is to add a desktop layout/route that imports the existing module components (CreateJob, PpcReviewSheet, JobDetail, Maintenance, Notifications) and renders them inside popups/drawers, with the live job board (GET /jobs) as the home surface. No backend changes are required.